NotebookLM Clone: End-to-End Project Report

Himanshi Shrivas

Executive summary

This project is a working NotebookLM-style app: multiple notebooks, PDF, PPTX, TXT and URL sources, cited RAG chat, and report and quiz generation, deployed to Cloud Run from GitHub. Stack: Gradio, ChromaDB, all-MiniLM-L6-v2 embeddings, a cross-encoder reranker and Groq's gpt-oss-120b. On a 40-question evaluation, cross-encoder reranking beat plain vector search on MRR@10 (0.871 vs 0.731).

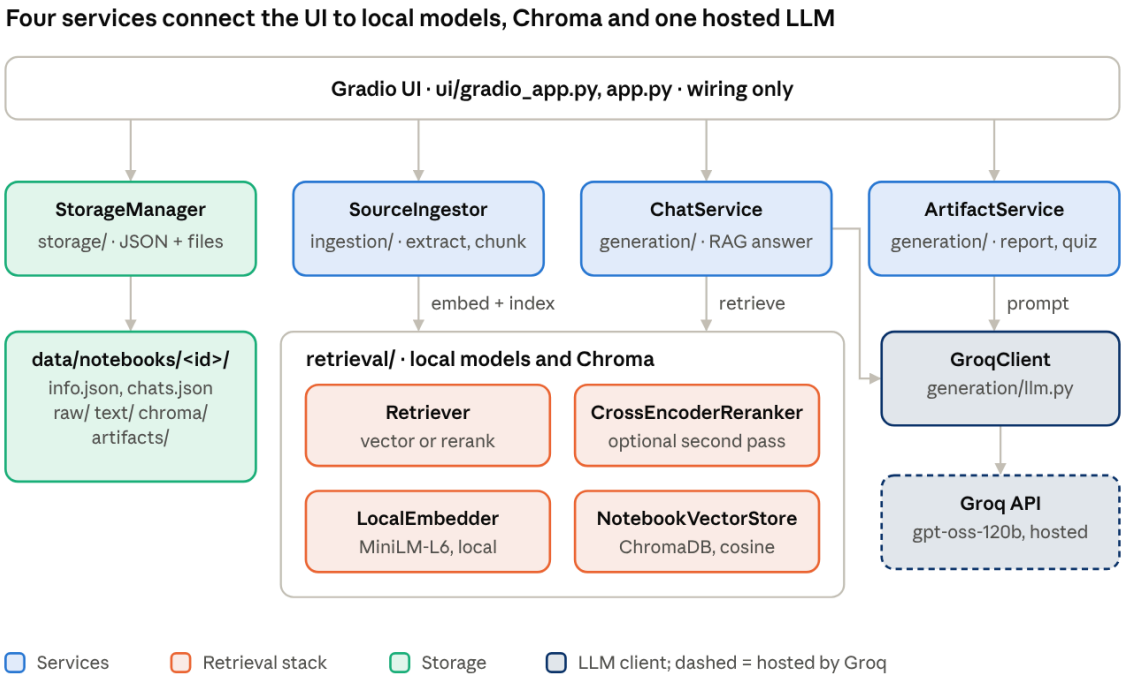
Problem, requirements and scope

The project is a NotebookLM-style app: users keep separate notebooks, add sources, chat with them with visible citations, and generate study artifacts.

Area	Requirements
Notebooks	Create, rename, delete, switch; unique ID; sources, chats and artifacts stored per notebook
Ingestion	PDF, PPTX, TXT and web URL: extract text, chunk, embed, store in the vector DB tagged with the notebook ID; keep source name, page/slide/URL and chunk index for citations
RAG chat	Question, retrieve, LLM, answer with citations; chat history persisted and reloadable
Artifacts	Report (.md) and Quiz (.md with answer key), viewable and downloadable
Storage	Survives restart; persistence limits documented if the host has them
UI	Gradio or similar: notebook manager, upload, URL ingest, chat, citations, artifact buttons, view/download, clear errors
RAG evaluation	Compare at least two retrieval approaches on a small question set; log question, method, retrieved chunks, response time, answer quality, conclusion
Deploy and CI/CD	Hosted app, secrets kept out of the repo, a GitHub push triggers deployment (GCP deployment is permitted; this project runs on Cloud Run, see the CI/CD section)
Deliverables	Repo with README, requirements, workflow, .gitignore and architecture doc; live URL; evaluation write-up; 1-2 minute demo

System architecture

A Gradio UI sits over four services; two of them share a local retrieval stack, and only the generation step reaches the hosted LLM.



System architecture · 4 services, 4 retrieval parts, 1 hosted LLM

Ingestion writes chunks into the notebook's Chroma collection, chat reads them back through the Retriever, and both text-generating services go through one GroqClient.

Module	Responsibility
`storage/`	Notebook CRUD, sources, chats and artifact metadata over the per-notebook folder; pure JSON and files
`ingestion/`	Extract text per page or slide, chunk (900 characters, 120 overlap), embed in batch, index in Chroma with citation metadata
`retrieval/`	LocalEmbedder, NotebookVectorStore (one Chroma collection per notebook, cosine), Retriever with `vector` and `rerank` methods
`generation/`	ChatService, ArtifactService and GroqClient
`ui/`	Gradio Blocks wiring only, no business logic
`eval/`	Two-method retrieval comparison: corpus, 40 labelled questions, metrics, paired bootstrap

Module	Responsibility
	statistics, and LLM-judge answer scoring with a human spot-check

Design decisions that shape the rest

- Local embeddings and reranker with a hosted LLM, so only generation needs a network key.
- Plain files plus a Chroma persistent client instead of a database: transparent and zero infrastructure.
- Lazy model loading, so the app and tests start fast.
- Per-page chunking, so every citation can point to a page or slide.
- LLM provider isolated in `generation/llm.py`, so switching providers is a one-file change.
- One folder and one Chroma collection per notebook, so retrieval can never cross notebooks.

Technology choices and tradeoffs

The stack keeps everything local and file-based except the LLM, so the app needs no database server and one secret. Each choice below buys simplicity and pays for it somewhere.

Choice	Why it was picked	Tradeoff accepted
Gradio UI	Python-only, no frontend build, enough for the notebook manager, chat, citations and artifact views	Less layout control than a custom web frontend
Groq-hosted gpt-oss-120b (replaced Gemini during development)	Hosted model, so no GPU to run; isolated in `generation/llm.py`, so the switch was a one-file change	The only network dependency and the only secret (`GROQ_API_KEY`); output is non-deterministic at temperature 0.2
all-MiniLM-L6-v2 embeddings, run locally	No API key, no per-call cost, no network at query time; small enough to bake into the image	A small model can miss hard paraphrases that a larger embedding model would catch
ChromaDB persistent client, one collection per notebook	Embedded, no server, cosine search with metadata for citations; separate collections make cross-notebook leakage impossible	Not built for multi-node scale; its files live on the container disk
Cross-encoder reranker (ms-marco-MiniLM-L-6-v2)	Reads question and passage together, so it scores relevance more precisely than vector similarity; it lifted MRR@10 from 0.731 to 0.871 in the evaluation	About 80 ms more per query (98 vs 20 ms in the evaluation), a second model of about 90 MB, and it can only reorder vector's top 20
Plain JSON and files for notebooks, sources and chats	Transparent, easy to inspect, zero infrastructure; the storage	No transactions or concurrent-writer safety, which

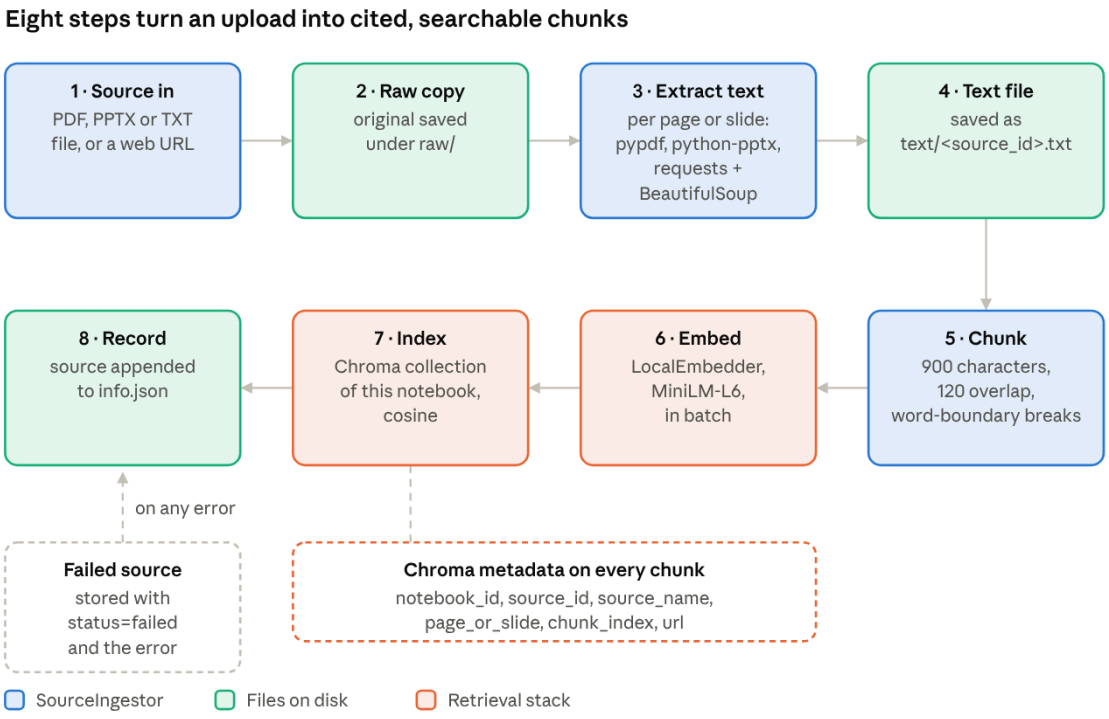
Choice	Why it was picked	Tradeoff accepted
	root is one constructor argument	is why the deploy runs a single instance
Per-page chunking, 900 characters with 120 overlap	Every citation can point to an exact page or slide; overlap protects sentences that straddle a chunk edge	Chunks never span pages, so an idea split across a page break is retrieved in two pieces
Lazy model loading	App and tests start fast	The first request after a cold start is slower
Cloud Run with CPU-only torch and both models baked into the image	Container deploy from GitHub Actions, scales to zero, no model download at start-up	Ephemeral disk, cold starts, and a larger image to build

Why local embeddings with a hosted LLM. Embedding runs on every ingest and every question, *so a local model removes a recurring cost and a failure poin*t. Generation is the one step that needs a large model, so only that step reaches over the network.

Why compare vector with rerank? The assignment asks for two retrieval methods. Vector search is the fast baseline; a cross-encoder is the standard precision upgrade. The evaluation section shows what the upgrade bought: nothing measurable on the 17-chunk pilot, but a clear MRR gain on the harder 112-chunk set, for about 80 ms more per query.

Ingestion pipeline

Ingestion turns a file or URL into chunks that each remember their page, so every answer can cite its source. A source that fails is saved with its error instead of being dropped.



Ingest pipeline · 8 steps, 1 failure branch

Steps 1 to 5 prepare text, steps 6 and 7 make it searchable, and step 8 registers the source in the notebook.

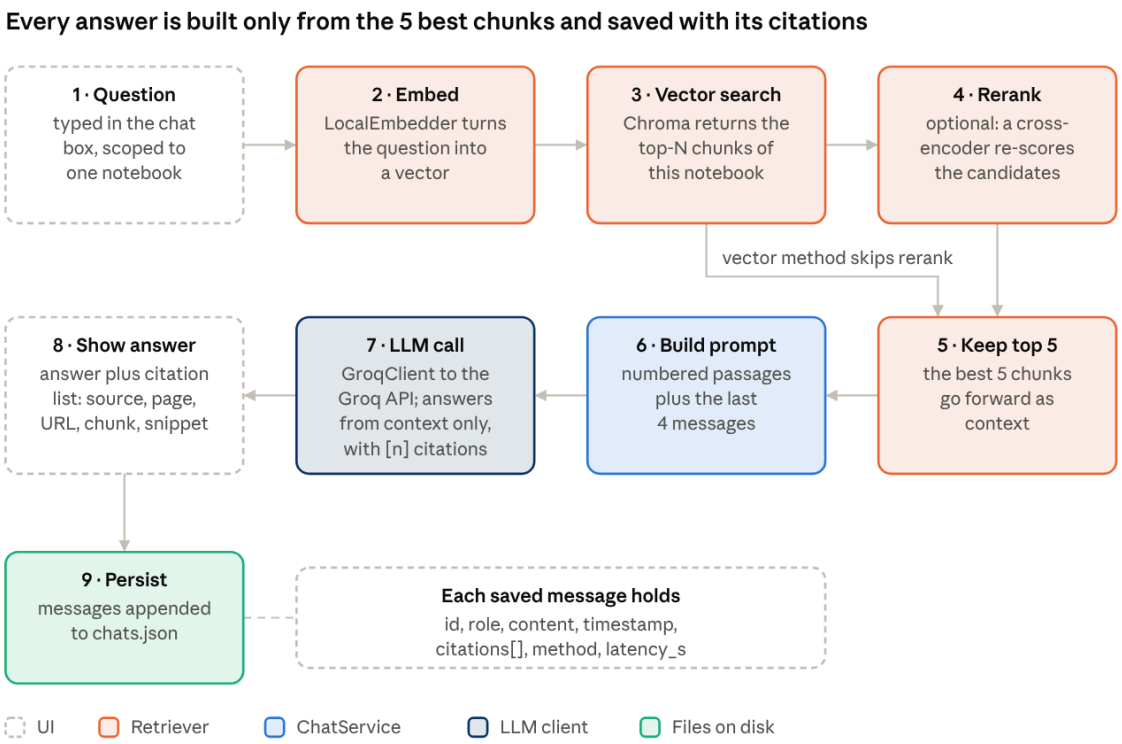
Source type	Extracted with	Citation points to
PDF	pypdf, per page	Page number
PPTX	python-pptx, per slide	Slide number
TXT	Plain text read	Source name and chunk
Web URL	requests + BeautifulSoup	The URL

Why chunk this way? Chunking per page keeps every citation precise. Smaller chunks sharpen retrieval but carry less context, and larger ones do the opposite; 900 characters with a 120-character overlap is a middle setting, and word-boundary breaks avoid cutting a word in half. Embedding in batch means one model call per group of chunks instead of one per chunk.

Limits- Scanned PDFs have no text layer and are not OCR'd, and pages built with JavaScript do not render, so both extract poorly.

RAG chat pipeline

A question is answered only from the notebook's own top 5 chunks, and the answer comes back with numbered citations that point to a source, page or slide, and snippet.



chat pipeline · 9 steps, 1 optional rerank branch

The top row is retrieval, with a branch: the `vector` method goes straight from search to step 5, while `rerank` adds the cross-encoder pass. The second row is generation and display; step 9 saves the exchange.

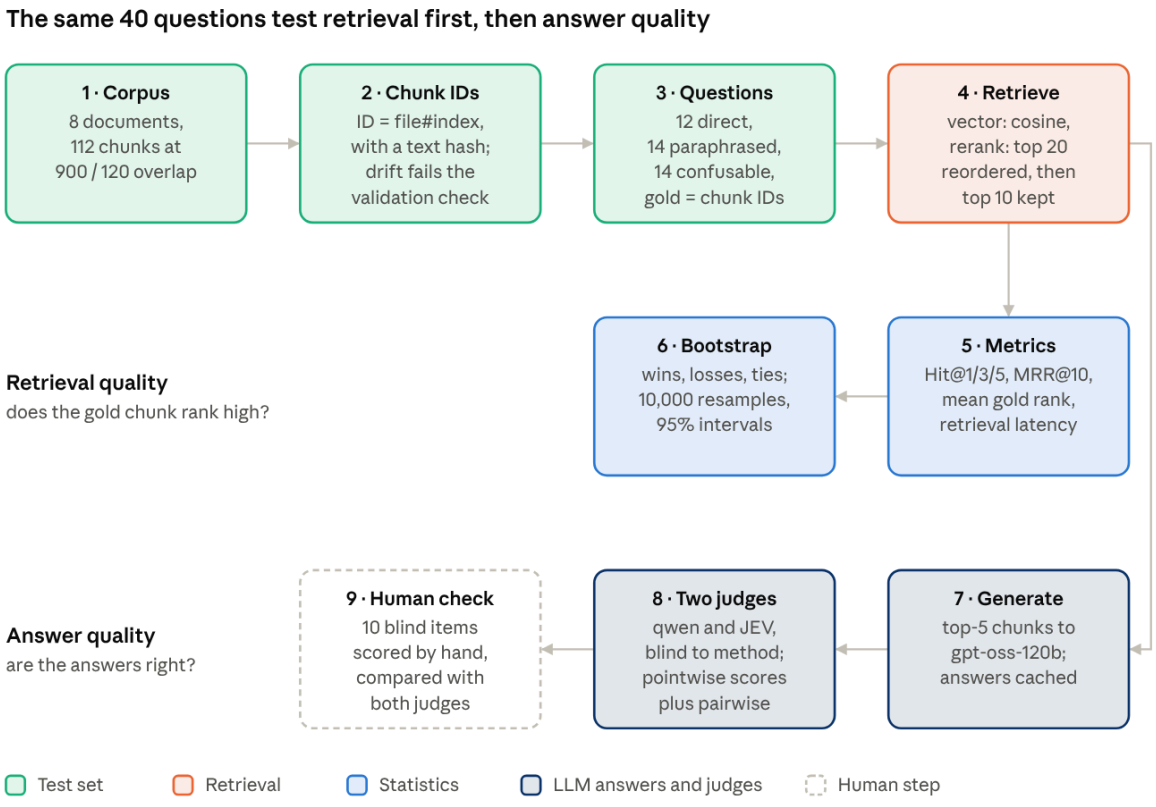
Method	How it works	Cost	Should win when
`vector`	Embed the question, take Chroma's cosine top results	Fastest: 19.8 ms mean retrieval in the 40-question evaluation	The notebook is small and the answer ranks near the top
`rerank`	Take Chroma's top candidates (20 in the evaluation), rescore each with the cross-encoder, keep the best	98.4 ms mean, plus a second model of about 90 MB	Several passages look alike: on the confusable questions MRR@10 rose from 0.673 to 0.917

Grounding. The prompt numbers each retrieved passage and tells the model to answer using only that context with `[n]` markers, which the UI turns into a citation list. In the 40-question evaluation, the answer cited a gold chunk for 37/40 questions with rerank and 28/40 with vector.

Tradeoffs. Sending only 5 chunks keeps the prompt small and the answer focused, but a fact held in a sixth chunk is invisible to the model. Including the last 4 messages lets follow-up questions work, while capping how much history the prompt carries.

RAG evaluation

On a harder 112-chunk corpus with 40 labelled questions, cross-encoder reranking beat plain vector search: MRR@10 rose from 0.731 to 0.871 (the 95% interval for the gain excludes 0), and the gold chunk reached the LLM's context for 40/40 questions against 34/40. Full write-up: [docs/EVALUATION.md](); raw data in `eval/results/v2/`. The earlier 18-question pilot could not separate the methods, so this round is the one to cite; the pilot is kept at the end of this section.



evaluation pipeline · 9 steps, 2 test lanes

The same 40 questions drive two tests: retrieval quality (steps 4 to 6) and answer quality (steps 7 to 9). The questions and gold labels were fixed before either method ran.

Test design

- **Corpus:** 8 documents, 112 chunks at 900 characters with 120 overlap: the 2 pilot documents plus 6 written for this evaluation in our own words (more Apollo missions, Mercury/Gemini, Skylab/ISS, Voyager, cellular respiration with mitosis/meiosis, C3/C4/CAM). They overlap on purpose (dates, crews, durations, enzymes, locations), so look-alike distractor passages exist.
- **Questions:** 40 in `eval/questions_v2.json`: 12 direct, 14 paraphrased (little word overlap with the source) and 14 confusable (a similar passage with a different value exists elsewhere, and the question names the one it wants). Written before either method was run and not edited afterwards.
- **Gold labels:** chunk IDs (`file#chunk_index`), not phrase matching. A chunk is gold if it contains the full answer-bearing sentence, so the 40 questions carry 56 gold IDs. Each label stores a sha256 of its chunk text, and `python -m eval.validate_questions` fails if the corpus or chunking changes.
- **Methods:** `vector` is Chroma cosine order. `rerank` takes the 20 nearest vector candidates and reorders them with the cross-encoder `ms-marco-MiniLM-L-6-v2`. Each returns a ranked top 10, and the top 5 go to the LLM, as in the app.
- **Generation:** Groq `openai/gpt-oss-120b`, no chat history, answers cached.
- **Metrics:** Hit@1/3/5, MRR@10, mean rank of the first gold chunk (a question with no gold chunk in the top 10 counts as a miss and is left out of that mean), and retrieval latency with the models warmed up first.

Retrieved chunks and response time

Every question's retrieved chunk IDs, scores, gold rank, answer and timings are logged in `eval/results/v2/eval_log.json`. Example, q01 'Who were the three crew members of Apollo 12?' (gold chunk `apollo_missions_extended.txt#004`):

Method	Gold chunk rank	Answer
vector	Not in the top 10 (top result: `apollo_missions_extended.txt#011`)	Said it could not find the names
rerank	2	Named Charles 'Pete' Conrad, Alan Bean and Richard Gordon, with a citation

Retrieval takes about 20 ms with vector and 98 ms with rerank. End to end, including the LLM call, a question takes 9.4 s with vector and 9.8 s with rerank, so the extra retrieval cost is small next to the LLM.

Retrieval results (n = 40)

Method	Hit@1	Hit@3	Hit@5	MRR@10	Mean gold rank	Missed top 10	Mean latency
vector	24/40	34/40	34/40	0.731	2.08	1	19.8 ms
rerank	31/40	40/40	40/40	0.871	1.32	0	98.4 ms

MRR@10 by question type (point estimates, 12 to 14 questions each):

Type	vector	rerank
direct (12)	0.792	0.861
paraphrased (14)	0.736	0.833
confusable (14)	0.673	0.917

Paired comparison (rerank minus vector)

Question by question, rerank was better on 13, worse on 5 and equal on 22 (reciprocal rank). On Hit@1 it won 11, lost 4 and tied 25. The intervals below come from a paired bootstrap over questions (10,000 resamples, fixed seed, 95% percentile interval).

Difference	Estimate	95% interval	Includes 0?
MRR@10	+0.140	[+0.023, +0.258]	No
Hit@1	+0.175	[+0.000, +0.350]	Borderline: the lower end is exactly 0.000, so we treat it as including 0

By question type (exploratory, not corrected for multiple comparisons): confusable MRR +0.243 [+0.053, +0.437] and Hit@1 +0.357 [+0.071, +0.643] exclude 0; the direct and paraphrased intervals include 0.

Does the right evidence reach the model?

Method	Gold chunk in context	Answer cites a gold chunk	Mean end-to-end latency
vector	34/40	28/40	9.4 s
rerank	40/40	37/40	9.8 s

These automatic signals show whether the right evidence reached the model and was cited, not whether the answer is correct. Answer quality is scored by the judges below.

Answer quality: two LLM judges and a human spot-check

Both methods' answers were scored by two judges that did not write the answers and never saw the method name: `qwen/qwen3.8-27b` on Groq (temperature 0) and TypeSafe JEV (`typesafe/jev-1.13`, via OpenRouter). The JEV judge is used by the eval scripts only; the app itself needs just `GROQ_API_KEY`. Pointwise scores are faithfulness (is each claim supported by the cited passages) and correctness against the reference answer, each 1 to 5. Pairwise shows both answers as A/B in a balanced, seeded order (seed 4242) and asks for a winner or a tie.

Judge	Method	Faithfulness (mean)	Correctness (mean)
qwen	vector	4.325	4.475
qwen	rerank	4.650	5.000
JEV	vector	4.393	4.453
JEV	rerank	4.607	4.936

Judge	rerank wins	vector wins	Ties	Identical answers (within ties)	Sign test p (two-sided, ties dropped)
qwen	9	3	28	5	0.146
JEV	7	1	32	5	0.070

- **Both judges agree on direction.** Rerank scored higher on faithfulness and correctness and won more pairwise comparisons than it lost. Correctness was 4.5 or higher for 40/40 rerank answers, against 34/40 (qwen) and 35/40 (JEV) for vector.
- **The pairwise result is suggestive, not significant.** Most comparisons were ties, and the sign test on the decided ones does not reach $p < 0.05$ for either judge. The pointwise differences were not given a paired interval.
- **It matches the retrieval result.** Vector put the gold chunk in the top-5 context for 34/40 questions and rerank for 40/40, and the judged gap is about that size. The cause was not isolated for each question.

Human spot-check (10 blind items). The author scored 10 randomly chosen answers, method hidden, with the same rubrics. Correctness agreed exactly with both judges (100%). Faithfulness agreed exactly for 80% of items (qwen) and 70% (JEV), and within one point for 90% with both. The author scored all 10 answers 5/5, so the sample holds no weak answers: it shows the judges do not under-rate good answers, not that they catch bad ones. The disagreements are faithfulness cases where a judge scored below 5, notably one answer that cited nothing valid, which the judge rubric penalises. The first qwen pass used an unbalanced A/B order; it is kept as `firstpass_qwen_unbalanced_order.json` and replaced by the balanced run above.

Conclusion

- **Reranking retrieved the gold chunk earlier on this harder set.** MRR@10 rose from 0.731 to 0.871 with an interval that excludes 0, and Hit@3 was 40/40 against 34/40.
- **The Hit@1 gain is suggestive, not established:** +0.175 with an interval that touches 0.
- **The gain is concentrated in confusable questions** (look-alike passages), which is where a cross-encoder reading question and passage together is expected to help. The per-type split is exploratory.
- **Cost:** about 80 ms extra retrieval latency per query in this run (98 vs 20 ms), negligible next to the roughly 9 s LLM call, plus a second model of about 90 MB.
- **Limit:** rerank only reorders vector's top 20, so it cannot recover a gold chunk outside them.
- **Judged answer quality points the same way** (correctness 4.94 to 5.00 vs 4.45 to 4.48 for vector), but the pairwise wins are not significant and judge-human agreement was checked on only 10 items.
- **Decision:** keep reranking as the default, with `vector` selectable in the UI. The evidence is consistent with a real retrieval improvement on this set, but it is not proof that it generalises.

Limitations

- 40 self-written questions over one small corpus (112 chunks). The corpus, questions and gold labels were all written by us, so they may reflect our expectations about what is hard.
- Gold means 'the chunk contains the answer sentence'. Overlap can duplicate a sentence in two neighbouring chunks (both then count as gold), and a few questions also count a later summary paragraph that restates the fact.
- Single run; latency comes from one pass on one CPU. Generation uses temperature 0.2, with answers cached.

- Answer quality rests on LLM judges, and the human spot-check covers only 10 items, all scored 5/5 by the author.

Reproduce

```
python -m eval.validate_questions        # check gold labels
python -m eval.run_eval_v2 --retrieval-only   # retrieval metrics, no LLM
python -m eval.run_eval_v2               # retrieval and generation
python -m eval.paired_stats              # bootstrap statistics
python -m eval.judge --judge both        # LLM judges (add --agreement for the
spot-check)
```

Judge calls are cached in `eval/results/v2/judge_cache.json`.

Pilot round (v1): 2 documents, 18 questions

Kept for reference: both methods scored Hit@3 = 18/18, so the pilot could not separate them, which is why the v2 round above was built. `eval/corpus/` now also holds the v2 documents, so rerunning `python -m eval.run_eval` no longer reproduces these numbers.

Plain vector search and vector plus cross-encoder rerank tied: both found the gold passage for all 18 test questions. Full log and write-up: docs/EVALUATION.md.

Setup

- Corpus: 2 short documents (Apollo program, photosynthesis) in one notebook, about 17 chunks of about 900 characters.
- Questions: 18 in `eval/questions.json`, 10 direct and 8 paraphrased with little word overlap with the source.
- `vector`: all-MiniLM-L6-v2 embeddings, Chroma cosine, top 3.
- `rerank`: Chroma top 20 candidates rescored by the ms-marco-MiniLM-L-6-v2 cross-encoder, top 3 kept.
- Same LLM (Groq gpt-oss-120b) and same prompt for both methods, so only retrieval differs. Run with `python -m eval.run_eval`.

Method	Hit@3	Mean rank of hit	MRR	Answer keyword coverage	Mean retrieval latency
vector	18/18	1.11	0.94	1.00	113 ms
rerank	18/18	1.11	0.96	0.97	142 ms

The methods disagreed on only two questions:

Question	vector rank	rerank rank
How much material did the first moonwalkers bring back to Earth?	2	1
What happens to plant sugar production when it gets extremely hot?	2	3

End to end, a question takes 5.5 s with vector and 5.6 s with rerank. The LLM call dominates, not retrieval.

Answer quality. The author read a sample of answers from both methods and judged them correct and grounded in the retrieved passages. All 36 answers carry `[n]` citations. The one keyword-coverage miss (rerank, 0.50) is a false negative: the answer says heat damages enzymes instead of using the word the check looked for.

Conclusion

- The methods are effectively tied here. Reranking helped one question and hurt one, moving MRR from 0.94 to 0.96, which is within noise at 18 questions.
- Reranking costs about 30 ms more per query on CPU, a second model download (about 90 MB) and extra memory.
- Why no clear win: with about 17 chunks, the top 3 almost always contains the answer. Reranking should matter more on larger corpora, where first-stage retrieval misses relevant chunks.
- Decision: keep rerank available and as the UI default, but this evaluation does not prove it is better.

Limitations. The corpus and questions are small and self-written. Gold-passage detection uses exact-phrase matching, which can undercount correct chunks phrased differently. Keyword coverage is a coarse proxy for quality. The LLM is non-deterministic (temperature 0.2).

Artifacts: report and quiz

The app turns a whole notebook into a Report or a Quiz with answer key, saved as Markdown and available to view and download in the UI. `ArtifactService` (`generation/`) does the work.

1. **Collect.** Take every chunk of the notebook.
2. **Sample.** Sample the chunks evenly, up to 30k characters, so a large notebook fits one prompt and every source is represented rather than only the first pages.
3. **Generate.** Send the sample to the LLM through `GroqClient` with a report prompt or a quiz prompt.
4. **Save.** Write the `.md` file into the notebook's `artifacts/` folder.
5. **Record.** Add the artifact to `info.json` (id, type, filename, path, created_at); the UI lists it for viewing and download.

Tradeoff. Even sampling to a 30k-character budget is simple and predictable, but it is not retrieval: it cannot focus on a topic, and very large notebooks are summarised from a thin slice. A map-reduce summary would cover more at several times the LLM calls, which the assignment does not require. Artifact quality and grounding were not formally evaluated.

Storage, data model and persistence limits

Each notebook is one folder, and the deployed app loses those folders whenever Cloud Run replaces its instance. Locally, everything survives a restart.

```
data/notebooks/<id>/
  info.json     notebook record: sources and artifacts metadata
  chats.json    persisted chat history
  raw/          original uploaded files
  text/         extracted text, one file per source
  chroma/       this notebook's vector index
  artifacts/    generated report and quiz .md files
```

File	Fields
`info.json`	id, name, created_at, updated_at; `sources[]` with id, type, name, path, status, metadata;

File	Fields
	`artifacts[]` with id, type, filename, path, created_at
`chats.json`	list of messages: id, role, content, timestamp, citations[], method, latency_s
Chroma metadata (per chunk)	notebook_id, source_id, source_name, page_or_slide, chunk_index, url

Isolation. Each notebook has its own folder and its own Chroma collection, so a query can never retrieve another notebook's chunks.

Storage layer. `StorageManager` (`storage/`) is a thin abstraction over the folder: notebook create, rename, delete, sources, chats and artifact metadata. Its storage root is a single constructor argument.

Persistence limits on Cloud Run. The container filesystem is ephemeral. `data/` survives while an instance lives and is wiped when it is replaced: scale-to-zero after idle, every deploy, and restarts. Notebooks are therefore best effort in the deployed app. `--max-instances 1` stops one user's notebooks from being split across instances.

Not implemented, by design. Mounting a Cloud Storage bucket (GCS FUSE volume) or Filestore at `data/` would make notebooks durable. Because the root is one constructor argument, either is a small change. The assignment allows documenting the limit instead of solving it, which was the right use of time for this deadline.

User interface

The UI is a Gradio app with one job: wire user actions to the services. It holds no business logic, so every behaviour can be tested without the browser.

Tab	What the user does
Notebooks	Create and switch notebooks (rename and delete are implemented in StorageManager)
Sources	Upload PDF, PPTX or TXT files, or add a web URL; a source that fails is stored with status=failed and its error message
Chat	Ask a question, choose vector or rerank retrieval, read the answer with `[n]` citations and a citation list (source, page or slide, URL, chunk, snippet); history is saved per notebook
Artifacts	Generate a report or a quiz, view it, download the `.md` file

The code is `ui/gradio_app.py` plus the `app.py` entry point (port 7860 locally, 8080 in Docker).

The UI ships three themes and shows a status message for each action, including clear errors for an unsupported file type, a bad URL or a missing notebook.

Tradeoff. Gradio gets a complete interface in little code. In exchange the layout is functional rather than polished.

CI/CD and deployment

A push to `main` tests, builds and deploys the app to Cloud Run with no manual step. Live app: notebooklm-clone on Cloud Run. GCP deployment is permitted for this assignment, so the app ships as a container to Cloud Run .

1. **Push** to `main` on GitHub starts `.github/workflows/deploy.yml`.
2. **Test.** GitHub Actions runs the 27 offline tests (`python -m pytest tests -q`).
3. **Build.** Docker builds the image with CPU-only torch and both models (embedder and reranker) baked in. The workflow retries the base-image pull to survive Docker Hub rate limits.
4. **Publish.** The image is pushed to Artifact Registry.
5. **Deploy.** `gcloud run deploy` updates the Cloud Run service: 2 GiB, 2 vCPU, max 1 instance, scale to zero.

Item	Where it lives
`GCP_SA_KEY`	GitHub secret, used by the workflow to authenticate to Google Cloud
`GCP_PROJECT_ID`, `GCP_REGION`	GitHub variables read by the workflow
`GROQ_API_KEY`	Set on the Cloud Run service; never in the repo
`GROQ_MODEL`	Optional; defaults to openai/gpt-oss-120b

Why these settings. Baking the models into the image means a cold start never downloads them. Max 1 instance keeps each notebook folder on one disk. Scale to zero means no cost while idle, paid for with a slow first request.

Tradeoff. Cloud Run gives a standard container pipeline and scale to zero. The costs are an ephemeral disk (see Storage) and cold starts. Run locally with Python 3.10 on port 7860, or with Docker on port 8080.

Testing, security, limitations and future work

The app is tested offline, keeps its one secret out of the repo, and has four known limits, all documented.

Testing. The 27 offline tests live in `tests/` and run with `python -m pytest tests -q` after installing `requirements-dev.txt`. The same suite gates every deploy in GitHub Actions. Models load lazily, so tests start fast.

Security. `GROQ_API_KEY` is set on the Cloud Run service and `GCP_SA_KEY` is a GitHub secret; neither is stored in the repo. `.env.example` documents the variables, and `.gitignore` excludes local secrets and `data/`.

Limitation	Effect	Fix if needed
Ephemeral disk on Cloud Run	Notebooks are lost when the instance is replaced	Mount a GCS FUSE bucket or Filestore at `data/` (small change)
No OCR	Scanned PDFs yield no text	Add an OCR step in ingestion
No JavaScript rendering	Web pages built client-side extract poorly	Use a headless browser fetcher

Limitation	Effect	Fix if needed
Cold starts	The first request after idle is slow	Keep a warm instance, at a cost
Small self-written evaluation	n = 40 on one 112-chunk corpus; the Hit@1 gain and the judge pairwise results are suggestive, not significant	A larger, independently written corpus and question set

Deliverables checklist

Deliverable	Status	Where
GitHub repo: source, README, requirements, workflow, .gitignore	Done	darpan02-cypher/NotebookLM-Gemini-Notebook-Clone (13 commits on main)
Architecture doc	Done	`docs/`ARCHITECTURE.md in the repo, with `SPEC.md` at the root
Hosted app URL	Done, on Google Cloud Run	notebooklm-clone on Cloud Run
RAG evaluation write-up	Done	docs/EVALUATION.md and `eval/results/v2/`
1-2 minute demo recording	Done	https://drive.google.com/file/d/1OjKShNCfbV5F18gHOCrOvQ7EXZkwP9WT/view?usp=sharing